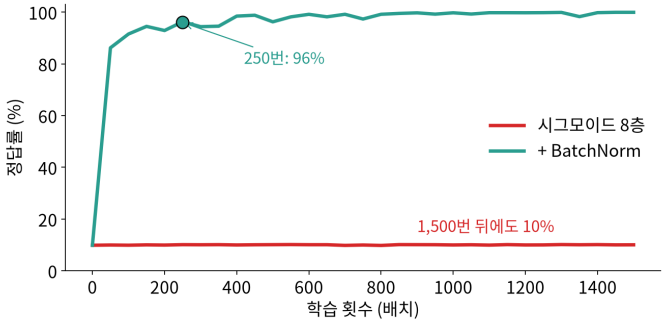
정규화 층: BatchNorm · LayerNorm (Normalization)

층마다 값을 평균 0 · 표준편차 1 로 맞추기: 어느 방향으로 맞출까

Machine Learning and Deep Learning

한국과학영재학교 (KSA)

오늘의 질문



시그모이드 8층: **1,500번 학습해도 10%** BatchNorm 을 끼우면 **250번에 96%**

무슨 일이 일어났나: 층을 지날수록 그림들이 똑같아진다

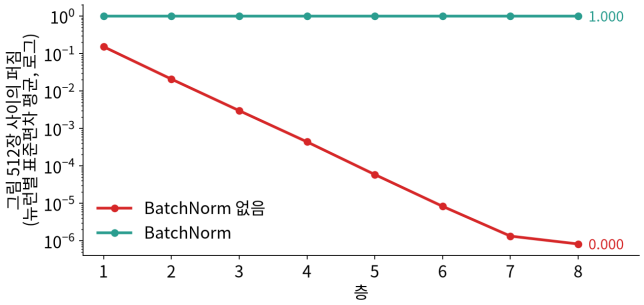


그림 512장이 받는 값의 퍼짐 (학습 전): 1층 0.15 → 8층 **0.0000002**

8층에는 모든 그림이 거의 같은 값 → 구별할 정보가 없다 BatchNorm: 층마다 다시 1 로

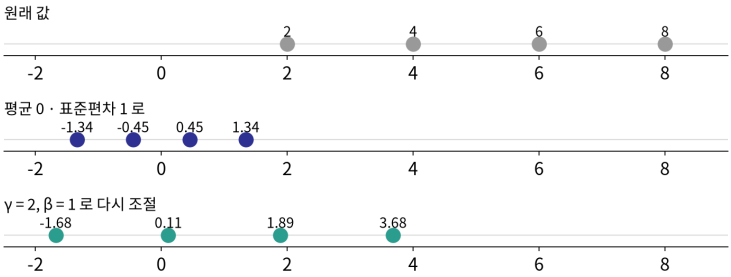
BatchNorm: 미니배치마다 평균 0, 표준편차 1 로

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i \qquad \sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}} \qquad y_i = \gamma\,\hat{x}_i + \beta$$

- m 미니배치 크기 뉴런마다 따로 계산한다
- ϵ 0 으로 나누지 않게 더하는 아주 작은 수 (예: 10^{-5})
- γ, β **학습하는** 크기 · 위치 (처음엔 1, 0)

손으로 한 번



배치 4개에서 받은 값 2, 4, 6, 8 $\mu_B = 5, \ \sigma_B^2 = \frac{9+1+1+9}{4} = 5$

$\hat{x} = \frac{x-5}{\sqrt{5}}$: $-1.34, -0.45, 0.45, 1.34$ $\gamma = 2, \beta = 1$: $-1.68, 0.11, 1.89, 3.68$

왜 γ, β 를 다시 곱하고 더하나

항상 평균 0 · 표준편차 1 로 묶어 두면 시그모이드의 **가운데 직선 부분**만 쓰게 된다

γ, β 를 학습하면 신경망이 **알맞은 크기와 위치를 스스로** 고른다

필요하면 $\gamma = \sigma_B$, $\beta = \mu_B$ $\rightarrow$ **원래 값으로 되돌릴 수도** 있다

그래서 BatchNorm 을 넣어도 신경망이 표현할 수 있는 것은 줄어들지 않는다

효과

우리 실험 (시그모이드 8층)	**10%** → 250번에 **96%**
논문 (2015, ImageNet)	같은 정확도를 **14배 적은** 학습으로
덤	큰 학습률도 안정 · 초기화에 덜 민감 · 약간의 정규화

왜 잘 되는지: 처음엔 “층 입력 분포의 출렁임 (internal covariate shift) 을 줄인다”고 설명했지만, 지금은 “손실 지형을 매끄럽게 만들어 큰 걸음을 가능하게 한다”는 설명이 더 받아들여진다

시험 때는 모아 둔 평균을 쓴다

시험 때는 그림이 **한 장씩** 들어올 수도 있다 $\rightarrow$ 배치 평균을 구할 수 없다

$$\mu_{\text{run}} \leftarrow 0.9\,\mu_{\text{run}} + 0.1\,\mu_B \qquad \sigma^2_{\text{run}} \leftarrow 0.9\,\sigma^2_{\text{run}} + 0.1\,\sigma^2_B$$

학습하면서 배치 평균 · 분산을 **조금씩 모아** 두었다가 시험 때 그 값을 쓴다

`model.train()` = 배치 통계 `model.eval()` = **모아 둔 통계**

어디에 넣나, 그리고 약점

위치	`Linear` (합성곱) → **BatchNorm** → 활성 함수
합성곱 (11주차)	채널마다 평균 · 분산 (`BatchNorm2d`) ResNet 의 기본 부품
약점	배치가 작으면 평균이 불안정, 순서 데이터에 어색 → 다음 쪽

같은 표, 다른 방향: BatchNorm vs LayerNorm

BatchNorm: 열마다 (샘플 4개끼리)

	특징1	특징2	특징3	특징4	특징5	특징6
A	1.0	1.4	0.6	-0.3	0.3	-0.5
B	1.1	3.0	0.3	0.1	1.7	1.5
C	1.2	-0.4	1.0	2.0	-1.0	0.3
D	-1.9	-0.9	-1.8	0.6	-0.9	1.4

LayerNorm: 행마다 (특징 6개끼리)

	특징1	특징2	특징3	특징4	특징5	특징6
A	1.0	1.4	0.6	-0.3	0.3	-0.5
B	1.1	3.0	0.3	0.1	1.7	1.5
C	1.2	-0.4	1.0	2.0	-1.0	0.3
D	-1.9	-0.9	-1.8	0.6	-0.9	1.4

BatchNorm: **샘플들끼리** (세로) LayerNorm: **특징들끼리** (가로)

A 의 LayerNorm: 평균 0.42, 표준편차 0.67 $\rightarrow$ $(1.0 - 0.42)/0.67 = \mathbf{0.87}$

Ba, Kiros & Hinton (2016) “Layer Normalization” 그다음 $\gamma\hat{x} + \beta$ 는 똑같다

옆 샘플을 바꾸면?

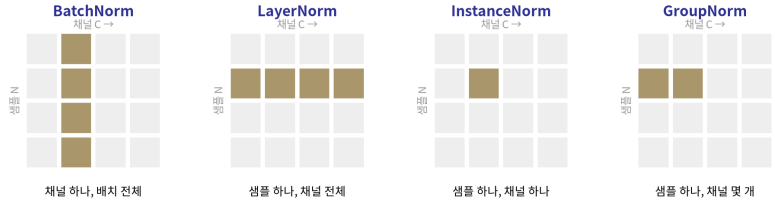
샘플 A 는 그대로 두고, 같은 배치의 B · C · D 만 다른 샘플로 바꿨다

A 의 첫 값 1.0 →	원래 배치	옆 샘플만 바꾼 배치	A 하나만
BatchNorm	0.50	**1.21**	**에러**
LayerNorm	0.87	**0.87**	**0.87**

BatchNorm: 결과가 **옆에 누가 있느냐**에 달렸다 LayerNorm: 자기 혼자 정한다

PyTorch `BatchNorm1d` 학습 모드에 샘플 1개 → “Expected more than 1 value per channel”

정규화 층 가족: 무엇끼리 평균을 내나



BatchNorm (2015)	CNN · ResNet (11주차)	사진을 많이 묶을 때
LayerNorm (2016)	트랜스포머 · GPT · BERT	길이가 제각각인 문장
InstanceNorm (2016)	그림 스타일 바꾸기	그림마다 밝기 · 대비를 지운다
GroupNorm (2018)	물체 탐지 · 분할	배치가 2 장이어도 그대로
RMSNorm (2019)	최근 언어 모델 (LLaMA 등)	평균 빼기 생략, 크기만 맞춘다

문제풀이

1. 배치 값 1, 3, 5, 7 을 BatchNorm 하면? (ϵ 무시, $\gamma = 1$, $\beta = 0$)
2. 위에서 $\gamma = 3$, $\beta = 10$ 이면 출력의 평균과 표준편차는?
3. 학습이 끝난 모델로 그림 한 장을 예측했더니 이상한 답이 나왔다. `model.train()` 상태였다. 왜 문제인가?
4. 배치 크기 1 로 학습하면 BatchNorm 은 어떻게 되나? LayerNorm 은?
5. 표 $\begin{pmatrix} 1 & 3 \\ 5 & 7 \end{pmatrix}$ (행 = 샘플, 열 = 특징) 의 BatchNorm 결과와 LayerNorm 결과는?

실습

1. BatchNorm 직접 구현 → 2, 4, 6, 8 손 계산 재현, `nn.BatchNorm1d` 와 비교
2. 시그모이드 8층: 층별 퍼짐과 학습 곡선, BatchNorm 있음 vs 없음
3. `train()` / `eval()`: 그림 한 장 예측과 모아 둔 평균 (`running_mean`)

Optical Recognition of Handwritten Digits (UCI; scikit-learn `load_digits`)

▶ Colab에서 실습 노트북 열기

정리

깊은 시그모이드: 층을 지날수록 그림들이 같은 값이 된다 (0.15 $\to$ 0.0000002)

BatchNorm 미니배치마다 뉴런별 평균 0 · 표준편차 1, 그다음 $\gamma\hat{x} + \beta$

10% $\to$ 250번에 96% 논문: 14배 적은 학습 큰 학습률도 안정

시험 때는 모아 둔 평균 · 분산 (`eval()`)

LayerNorm 샘플 하나 안에서: 옆 샘플과 무관 (0.87 그대로) $\to$ 트랜스포머

무엇끼리 평균을 내나: Batch · Layer · Instance · Group · RMS